

ORANGE PAPER · 橙皮书

Claude Code Skills 橙皮书

用技能扩展AI编程的边界

版本：v1.0

作者：滔哥

为谁创建：Claude Code 用户、AI 工具开发者、技术团队负责人

基于：Claude Code Skills 生态（2026年5月）

最后更新：2026年5月4日

目录

Part 1: 认识 Skills

- §01 Skills 是什么
- §02 Skills 的文件结构
- §03 Skills 的安装和管理
- §04 第一个 Skill: 10分钟上手

Part 2: 使用 Skills

- §05 Skills 的触发机制
- §06 Skills 的三级渐进式加载
- §07 常用内置 Skills 详解
- §08 社区 Skills 生态

Part 3: 创建 Skills

- §09 SKILL.md 的编写规范
- §10 前置脚本 (Preamble)
- §11 参考文件 (References)
- §12 打包发布

Part 4: 进阶技巧

- §13 条件触发和上下文感知
- §14 多 Skills 协作
- §15 Skills 的性能优化
- §16 Skills 和 Agent、MCP 的配合

Part 5: Skills 生态全集

- §17 开发类 Skills
- §18 设计类 Skills
- §19 运维类 Skills
- §20 效率类 Skills

Part 6: 实战场景

§21 场景一：用 Skills 搭建完整开发流程

§22 场景二：从零构建一个 Skill

§23 场景三：团队协作中的 Skills

附录

附录 A：SKILL.md 模板库

附录 B：常见问题和解决方案

附录 C：Skills 生态资源

附录 D：Skill 开发清单

Claude Code Skills 橙皮书

用技能扩展AI编程的边界

版本: v1.0

作者: 滔哥

为谁创建: Claude Code 用户、AI 工具开发者、技术团队负责人

基于: Claude Code Skills 生态 (2026年5月)

最后更新: 2026年5月4日

适用场景: 技能开发、工作流定制、团队效率提升、AI 编程进阶

前言

2026年4月，我在一个项目里用了30多个 Skills。

不是因为我想收集，是因为每个 Skill 真的能解决一个问题。有的帮我写测试，有的帮我做设计，有的帮我审查代码安全。以前这些事情要么手动做，要么写一堆脚本。现在一个 SKILL.md 文件就搞定了。

Claude Code Skills 是什么？一句话：给 Claude Code 装技能包。

你的 Claude Code 默认什么都会一点，但什么都不精。装上 Skill 之后，它在某个领域突然变强了。就像游戏里给角色装装备——本体能力不变，但输出完全不同。

这本书是我基于 Claude Code Skills 生态的实际使用经验写的深度手册。它不是功能列表的翻译，而是告诉你怎么用 Skills、怎么写 Skills、怎么用 Skills 构建完整的工作流。

这本书能帮你做到什么：

- 30分钟上手 Claude Code Skills，安装并使用第一个 Skill
- 1天内掌握 Skills 的核心机制，知道什么时候触发、怎么加载
- 1周内学会自己写 Skills，把你的工作流变成可复用的技能包
- 1个月内用 Skills 构建完整的开发流程，从编码到部署

这本书不适合谁：

- 想找"开箱即用完美方案"的人——Skills 需要你根据项目调整

- 不愿意写 SKILL.md 的人——最好的 Skill 是自己写的
- 把 AI 当万能的人——Skills 是工具，不是魔法

好，我们开始。

阅读指南

时间	章节	目标
Day 1	§01-§04	理解 Skills，安装并使用第一个 Skill
Day 2	§05-§08	掌握触发机制，熟悉内置和社区 Skills
Day 3	§09-§12	学会创建自己的 Skills
Day 4-5	§13-§16	进阶技巧，多 Skills 协作
Day 6	§17-§20	浏览 Skills 生态全集
Day 7	§21-§23	实战场景，综合运用

Part 1: 认识 Skills

从零开始。读完这部分，你会知道 Skills 是什么、能做什么、跟其他扩展方式的本质区别是什么。

§01 Skills 是什么

01.1 一个真实的场景

2026年3月。我在做一个 React 项目。写完功能代码，该写测试了。

以前的流程：打开 Jest 文档 → 复制模板 → 改改改 → 跑一下 → 报错 → 再改。一轮下来20分钟。

那天我试了一下项目里的一个 Skill。在 Claude Code 里输入 `/tdd`。Claude 自动读取了我的代码，理解了组件结构，生成了完整的测试文件。跑一下，过了。

3分钟。

这就是 Skills 的价值。不是让 AI 更聪明，是让 AI 更专注。

01.2 Skills 的本质

Skills 本质上是一个指令包。它告诉 Claude Code：

- 你是谁（角色定位）
- 你要做什么（任务描述）
- 你怎么做的（工作流程）
- 你用什么工具（脚本、参考文件）

没有 Skills，Claude Code 是一个什么都会的通才。

装上 Skills，Claude Code 是某个领域的专家。

Claude Code + Skills = 通才 + 专业装备

01.3 Skills vs 其他扩展方式

扩展方式	本质	适用场景	复杂度
CLAUDE.md	全局指令	项目规范、通用规则	低
Skills	领域专精	特定任务的工作流	中
Agents	任务执行	多步骤自动化	高
MCP Server	外部能力	访问外部系统	高

重点看第三列。

CLAUDE.md 适合写"代码风格用2空格"这种全局规则。Skills 适合写"写测试时先分析代码结构再生成"这种专业工作流。Agent 适合做"从代码到部署"这种多步骤任务。MCP Server 适合连飞书、GitHub 这种外部系统。

它们不冲突，可以叠加。一个项目里，CLAUDE.md 定规范，Skills 定专业流程，Agent 做自动化。

01.4 核心价值

Skills 给 Claude Code 带来的三个核心价值：

- 第一，专注。** 没有 Skills 的时候，Claude Code 每次都要从零理解你的需求。装上 Skill，它已经知道该怎么做。
- 第二，复用。** 你花一下午调好的工作流，打包成 Skill，下次直接用。团队其他人也能用。

第三，标准化。10个人用同一个 Skill，产出质量一致。不会出现"小王写得好，小李写得差"的情况。

滔哥的经验：

我一开始觉得 Skills 是"高级用户才需要的东西"。用了一个月后发现，越是日常任务越需要 Skills。因为日常任务重复度高，一个好 Skill 能省下的时间是指数级的。

§01讲了"为什么"。§02讲 Skills 的文件结构——一个 Skill 到底长什么样。

§02 Skills 的文件结构

02.1 一个 Skill 的最小组成

一个 Skill 最少只需要一个文件：

```
my-skill/  
└─ SKILL.md
```

就这么简单。SKILL.md 是 Skill 的核心。没有它，Skill 不存在。

02.2 SKILL.md 的两部分

SKILL.md 由两部分组成：

```
---  
name: my-skill  
description: 这个 Skill 做什么  
---  
  
# 这里是 Skill 的正文  
  
具体的工作流程、规则、模板...
```

第一部分：YAML 前置元数据 (frontmatter)

用 `---` 包裹，定义 Skill 的基本信息。

字段	必填	说明
name	是	Skill 的名称，用于 <code>/name</code> 触发
description	是	一句话描述，Claude 用它判断什么时候触发
model	否	指定使用的模型（haiku/sonnet/opus）
user-invocable	否	是否可以被用户直接调用（默认 true）

第二部分：Markdown 正文

告诉 Claude 具体怎么做。这是 Skill 的灵魂。

02.3 完整的 Skill 目录结构

一个完整的 Skill 可以包含更多内容：

```
my-skill/
├── SKILL.md           # 核心指令（必需）
├── scripts/          # 可执行脚本
│   ├── preamble.sh   # 前置脚本（每次触发先运行）
│   ├── build.sh      # 构建脚本
│   └── test.sh       # 测试脚本
├── references/       # 参考文档
│   ├── patterns.md   # 代码模式
│   └── examples/     # 示例代码
├── assets/           # 静态资源
│   ├── template.html # 模板文件
│   └── config.json   # 配置文件
├── _metadata.json    # 元数据（版本、作者等）
└── tools.yml         # 工具声明
```

不是每个 Skill 都需要这些。简单的工作流只要 SKILL.md。复杂的工具型 Skill 才需要脚本和资源。

02.4 各目录的作用

目录	作用	什么时候用
SKILL.md	定义 Skill 的行为	必须
scripts/	运行时执行的脚本	需要调用外部命令时
references/	提供给 Claude 的参考文档	需要 Claude 参考模式或示例时
assets/	模板和静态资源	需要生成文件时
_metadata.json	Skill 的版本和元信息	发布到社区时
tools.yml	声明需要的工具	Skill 依赖外部工具时

02.5 三级渐进式加载

这是 Skills 最精妙的设计。

```
Level 1: 元数据 (始终在上下文中)
  ↓ 需要时加载
Level 2: SKILL.md 正文 (触发时加载)
  ↓ 需要时加载
Level 3: 脚本和参考文件 (按需加载)
```

Level 1 是 description 字段。它始终在 Claude 的上下文里，但只有一句话。Claude 用它判断"这个 Skill 跟当前任务相关吗"。

Level 2 是 SKILL.md 的正文。当 Claude 判断相关，或者用户直接输入 `/skill-name` 时，正文被加载。

Level 3 是脚本和参考文件。只有 SKILL.md 里明确引用时才加载。

这个设计很关键。因为上下文窗口有限。如果所有 Skill 的全部内容都加载，Claude 的上下文早就爆了。三级加载保证了：只加载需要的，忽略不需要的。

核心建议：

写 Skill 时，把最重要的信息放在 SKILL.md 正文里。参考文件放 Level 3。不要把所有东西都塞进 SKILL.md——它会被优先加载，太大了浪费上下文。

§02讲了 Skill 长什么样。§03讲怎么安装和管理。

§03 Skills 的安装和管理

03.1 三种安装方式

方式	命令	适用场景
npm	<code>npx skills add</code>	社区 Skills
插件	<code>/plugin install</code>	插件市场 Skills
手动	<code>git clone</code> + 复制目录	自定义 Skills

03.2 npm 安装

最简单的方式。

```
npx skills add nuwa-skill
```

这条命令会：

- 1. 从 npm 下载 Skill 包
- 2. 复制到项目的 `.claude/skills/` 目录
- 3. 自动注册到 Claude Code

安装完成后，输入 `/nuwa-skill` 就能触发。

03.3 插件安装

如果 Skill 发布在 Claude Code 的插件市场：

```
/plugin install huashu-design
```

03.4 手动安装

最灵活的方式。适合你自己写的 Skill，或者从 GitHub 克隆的 Skill。

```
# 克隆仓库
git clone https://github.com/example/awesome-skill.git

# 复制到 skills 目录
cp -r awesome-skill .claude/skills/

# 或者直接在 .claude/skills/ 下创建
mkdir -p .claude/skills/my-skill
```

03.5 Skills 的存放位置

Skills 可以放在三个地方：

位置	路径	作用域
项目级	项目/ <code>.claude/skills/</code>	当前项目
用户级	<code>~/.claude/skills/</code>	所有项目
全局级	npm / 插件市场	跨项目共享

项目级放项目特有的 Skills。比如这个项目的测试规范、部署流程。

用户级放你个人通用的 Skills。比如你写代码的习惯、常用的 prompt 模板。

全局级放社区共享的 Skills。经过社区验证，质量有保证。

03.6 查看已安装的 Skills

```
ls .claude/skills/  
ls ~/.claude/skills/
```

每个目录就是一个 Skill。目录名就是 Skill 名。

03.7 卸载 Skills

```
rm -rf .claude/skills/my-skill
```

就这么直接。删除目录就卸载了。

滔哥的经验：

我的习惯是：先装几个社区 Skills 跑通流程，然后开始自己写。社区 Skills 质量参差不齐，自己的才是最合手的。别怕写 SKILL.md，它比你想象的简单。

§03讲了安装。§04我们来做第一个项目——安装一个 Skill 并使用它。

§04 第一个 Skill：10分钟上手

04.1 你需要什么

- Claude Code 已安装并可用
- 一个项目目录

预计时间：10分钟。

04.2 安装一个社区 Skill

打开终端，进入你的项目目录：

```
cd your-project  
npm skills add alirezarezvani/claude-skills/skills/code-reviewer
```

如果 npm 不行，手动克隆：

```
git clone https://github.com/alirezarezvani/claude-skills.git /tmp/claude-skill  
mkdir -p .claude/skills  
cp -r /tmp/claude-skills/skills/code-reviewer .claude/skills/
```

04.3 查看 Skill 内容

```
cat .claude/skills/code-reviewer/SKILL.md
```

你会看到类似这样的结构：

```
---
name: code-reviewer
description: Review code for best practices, bugs, and improvements
---

# Code Reviewer Skill

When reviewing code, follow this process:

1. **Read the code** - Understand what it does
2. **Check for bugs** - Look for common patterns
3. **Review style** - Check naming, structure
4. **Suggest improvements** - Actionable feedback
...
```

04.4 使用 Skill

打开 Claude Code：

```
claude
```

在 Claude Code 中输入：

```
/review
```

或者描述你的需求：

```
帮我审查一下 src/index.ts 的代码质量
```

Claude 会自动触发 code-reviewer Skill，按照 Skill 定义的流程审查你的代码。

04.5 看看输出

Skill 会按以下流程输出：

1. 读取代码文件
2. 分析代码结构
3. 列出发现的问题
4. 给出改进建议

每个步骤都有清晰的说明。比直接让 Claude "帮我看看代码" 专业得多。

04.6 回顾

10分钟，从安装到使用。跑通了这个流程，你就知道 Skills 是怎么回事了。

关键收获：

- Skills 就是一个目录，里面有一个 SKILL.md
- SKILL.md 定义了 Claude 的工作流程
- 安装就是复制目录，使用就是输入 `/skill-name`

§04完成了第一个项目。§05开始深入 Skills 的触发机制——Claude 怎么知道该用哪个 Skill。

核心建议：

第一个 Skill 不要自己写。先用社区的，跑通流程，理解 Skills 是怎么工作的。然后再开始自己写。站在别人肩膀上，比从零开始效率高。

Part 2: 使用 Skills

掌握 Skills 的触发和加载机制。读完这部分，你会知道 Skills 什么时候启动、怎么加载、怎么跟 Claude 的上下文配合。

§05 Skills 的触发机制

05.1 两种触发方式

触发方式	示例	谁决定
用户显式触发	<code>/review</code> 、 <code>/tdd</code>	用户
Claude 自动触发	"帮我写测试" → 自动加载 tdd Skill	Claude

用户显式触发最可靠。你输入 `/skill-name`，Claude 直接加载对应的 SKILL.md。

Claude 自动触发更智能。Claude 根据你的描述和 Skill 的 description 字段，判断哪个 Skill 最相关。但有时候会判断错。

05.2 description 字段的重要性

description 是 Skills 触发的关键。Claude 用它来判断"这个 Skill 跟当前任务相关吗"。

```
# 好的 description
description: Review code for best practices, bugs, and security issues

# 不好的 description
description: A tool for code
```

好的 description 具体、明确。不好的 description 太模糊，Claude 不知道什么时候该触发。

05.3 显式触发的写法

```
/review          # 最简洁
/review src/main.ts # 带参数
/skill:review     # 带前缀 (如果配置了 SKILL_PREFIX)
```

05.4 自动触发的条件

Claude 自动触发 Skill 需要满足：

1. Skill 的 description 跟当前任务相关
2. Skill 的 user-invocable 不是 false
3. 当前没有其他 Skill 正在执行

如果你不想让某个 Skill 被自动触发：

```
---
name: my-private-skill
description: 只有我知道的 Skill
user-invocable: false
---
```

05.5 冲突处理

如果多个 Skill 的 description 都跟当前任务相关，Claude 会选最相关的一个。如果分不清，Claude 会问你。

核心建议：

description 写得越具体，自动触发越准确。"Write unit tests using Jest with snapshot testing" 比 "Write tests" 好10倍。

§06 Skills 的三级渐进式加载

06.1 为什么要渐进式加载

Claude 的上下文窗口有限。如果所有 Skill 的全部内容都加载进来，上下文早就爆了。

三级加载解决了这个问题：只加载需要的，跳过不需要的。

06.2 Level 1: 元数据层

始终在上下文中

每个 Skill 的 name 和 description 始终在 Claude 的上下文里。这一层极小，几乎不占空间。

Claude 用这一层做快速判断："这个 Skill 跟当前任务有关吗？"

06.3 Level 2: SKILL.md 正文

触发时加载

当 Claude 判断相关，或者用户直接输入 `/skill-name` 时，SKILL.md 的正文被加载到上下文中。

这一层包含 Skill 的核心指令：工作流程、规则、模板。

06.4 Level 3: 脚本和参考文件

按需加载

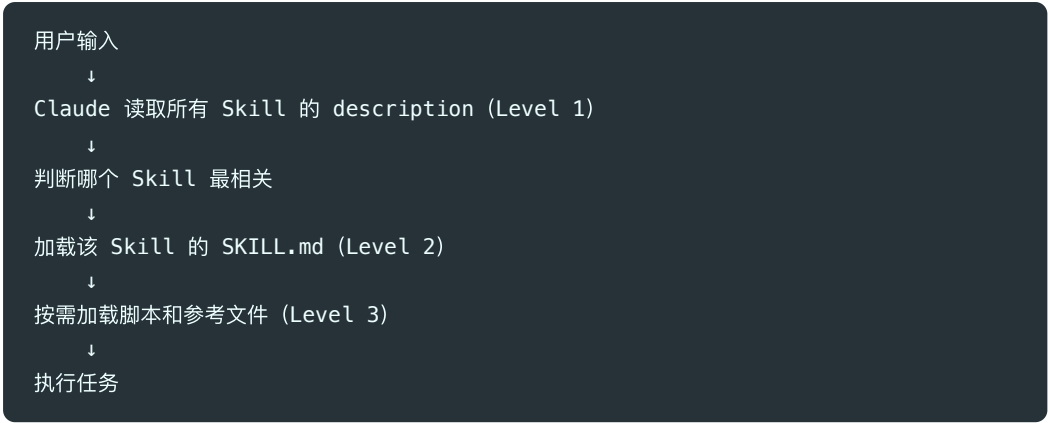
SKILL.md 里可以引用脚本和参考文件。只有当 Claude 执行到需要它们的步骤时，才会加载。

参考文件

在写测试前，先读取 ``references/test-patterns.md`` 了解项目的测试模式。

Claude 读到这句话时，才会去加载 `references/test-patterns.md`。

06.5 加载流程图



06.6 上下文管理策略

策略	做法	效果
description 精简	一句话说清楚	Level 1 极小
SKILL.md 聚焦	只放核心指令	Level 2 适中
参考文件分离	模式和示例放 references/	Level 3 按需
脚本外部化	复杂逻辑用脚本	不占上下文

滔哥的经验：

我见过有人把整个 API 文档塞进 SKILL.md。结果每次触发 Skill，上下文就占了一大半。正确做法是把文档放到 references/ 目录，SKILL.md 里只写"需要时读取 references/api-docs.md"。加载时机由 Claude 决定。

§07 常用内置 Skills 详解

07.1 什么是内置 Skills

Claude Code 本身带了一些基础能力。这些不算严格意义上的 Skills（没有 SKILL.md 文件），但它们的行为模式跟 Skills 一样：触发 → 加载指令 → 执行。

07.2 /commit

用途：智能提交代码

工作流：

- 1. 分析当前改动（git diff）
- 2. 自动生成规范的 commit message

3. 执行 git commit

跟直接 git commit 的区别：它会分析你的改动，生成有意义的 commit message。不是"fix bug"这种废话，而是"fix: resolve race condition in user auth middleware"这种有用的信息。

07.3 /review

用途：代码审查

工作流：

1. 读取代码文件
2. 检查代码风格、潜在 bug、安全问题
3. 给出具体的改进建议

适合场景：提交前自查、PR 前检查。

07.4 /help

用途：获取帮助

工作流：

1. 列出所有可用的 Skills
2. 显示每个 Skill 的简短说明
3. 给出使用建议

07.5 /clear

用途：清除上下文

工作流：

1. 清除当前对话的上下文
2. 保留项目配置

什么时候用：上下文太长，Claude 开始"忘事"的时候。

核心建议：

内置 Skills 是起点，不是终点。用它们理解 Skills 的工作方式，然后开始自己写。
自己写的 Skill 才是最适合你的。

§08 社区 Skills 生态

08.1 生态概览

2026年5月，Claude Code Skills 生态已经相当成熟：

指标	数据
社区 Skills 数量	658+
热门仓库 Stars	13,600+ (alirezarezvani/claude-skills)
覆盖领域	开发、设计、运维、文档、测试

08.2 主要来源

来源	说明	Stars
alirezarezvani/claude-skills	235个生产级 Skills	13.6K
nuwa-skill	思维蒸馏 Skill	17K
huashu-design	HTML 原生设计 Skill	11.7K
npm skills 包	分散的独立 Skills	–
个人仓库	开发者自用 Skills	–

08.3 Skills 分类

按用途分类：

类别	数量	典型 Skill
代码质量	50+	code-reviewer, linter, formatter
测试	40+	tdd, unit-test, e2e-test
设计	30+	huashu-design, ui-builder
文档	30+	doc-generator, readme-writer
部署	20+	deploy, ci-cd, docker
效率	60+	commit, refactor, optimize
AI 增强	40+	nuwa-skill, prompt-optimizer

08.4 怎么选择社区 Skills

选 Skill 的三个标准：

1. **Stars 数**：超过 100 的一般靠谱
2. **最近更新**：3个月内有更新的
3. **SKILL.md 质量**：结构清晰、说明具体

08.5 社区 Skills 的局限

社区 Skills 有几个问题：

- **质量参差不齐**：有些是半成品
- **不一定适合你的项目**：通用 vs 项目特定
- **可能过时**：Claude Code 更新后，旧 Skill 可能不兼容

滔哥的经验：

我装过30多个社区 Skills。真正长期在用的不到5个。原因很简单：社区 Skills 是通用的，我的项目是特定的。最好的策略是：用社区 Skills 学习怎么写，然后自己写适合自己的。

Part 3: 创建 Skills

从使用者变成创造者。读完这部分，你能写出自己的 Skills，并发布到社区。

§09 SKILL.md 的编写规范

09.1 基本结构

```

---
name: my-skill
description: 一句话说明这个 Skill 做什么
---

# Skill 标题

## 什么时候用

描述触发场景。

## 工作流程

1. 第一步做什么
2. 第二步做什么
3. 第三步做什么

## 规则

- 规则一
- 规则二

## 参考

需要读取的文件或资源。

```

09.2 YAML 前置元数据

```

---
name: tdd # Skill 名称 (必填)
description: > # 触发描述 (必填)
  Write tests first, then code.
  Use Jest for unit tests.
model: sonnet # 指定模型 (可选)
user-invocable: true # 用户可调用 (可选, 默认 true)
---

```

name: Skill 的唯一标识。用户通过 `/name` 触发。用小写字母和连字符。

description: 最关键的一句话。Claude 用它判断什么时候触发。写得越具体越好。

model: 可以指定 Skill 使用哪个模型。haiku 最快最便宜，opus 最强最贵。

09.3 正文编写技巧

技巧一：用指令式语气

```

# 好
读取代码文件，分析函数签名，生成测试用例。

# 不好
这个 Skill 会读取代码文件，然后分析函数签名，最后生成测试用例。

```

Claude 理解指令比理解描述更准确。

技巧二：用编号列表定义流程

1. 读取目标文件
2. 分析导出的函数
3. 为每个函数生成测试
4. 运行测试确认通过

编号列表告诉 Claude："按这个顺序执行。"

技巧三：用规则约束行为

- ```
规则
```
- 测试文件放在 `\_\_tests\_\_` 目录
  - 测试函数名用 `should` 开头
  - 不要 mock 数据库

规则比建议更有约束力。Claude 会更严格地遵守。

## 技巧四：用示例说明期望

```
示例
```

输入：`src/utils.ts` 包含 `add(a, b)` 函数

期望输出：

```
// __tests__/utils.test.ts

describe('add', () => {

 it('should add two numbers', () => {

 expect(add(1, 2)).toBe(3);

 });

});
```

示例比描述更直观。Claude 看到示例后，产出质量明显更高。

```
09.4 description 的写法
```

## 好的 description

---

description: >

Generate comprehensive unit tests for TypeScript functions.

Uses Jest framework with snapshot testing support.

# 不好的 description

description: "A testing tool"

```
好的 description 包含：
- 做什么 (Generate unit tests)
- 技术栈 (TypeScript, Jest)
- 特色 (snapshot testing support)

09.5 常见错误

错误	问题	正确做法
description 太短	Claude 不知道什么时候触发	写清楚做什么、怎么做
正文太长	浪费上下文	核心指令放正文，细节放 references/
没有示例	Claude 理解偏差	给一个输入输出示例
没有规则	行为不可控	用规则约束关键行为

> **核心建议**：
>
> 写 SKILL.md 的时候，想象你在带一个新同事。你会怎么告诉他做这件事？用同样的方式写。不要假设

§10 前置脚本 (Preamble)

10.1 什么是 Preamble

Preamble 是 Skill 触发时**最先运行**的脚本。它在 Claude 开始执行任务之前运行，用来收集信息
```

用户触发 Skill → 运行 Preamble → Claude 读取输出 → 执行任务

```
10.2 Preamble 的用途

用途	示例
收集环境信息	当前分支、Git 状态、文件列表
检查依赖	工具是否安装、配置是否正确
设置变量	定义路径、版本号等
预处理	生成临时文件、清理缓存

10.3 怎么写 Preamble

在 SKILL.md 中引用脚本：
```

## Preamble

---

Run this script before starting:

```
#!/bin/bash
_BRANCH=$(git branch --show-current)
_STATUS=$(git status --short)
echo "Branch: $_BRANCH"
echo "Status: $_STATUS"
```

Claude 执行 Skill 时，会先运行这个脚本，读取输出，然后开始任务。

### 10.4 实际示例

一个代码审查 Skill 的 Preamble:

```
#!/bin/bash
```



## 收集项目信息

---

```
echo "PROJECT: $(basename $(pwd))"

echo "BRANCH: $(git branch --show-current 2>/dev/null || echo 'not a git repo')"
```

```
echo "CHANGED_FILES: $(git diff --name-only HEAD 2>/dev/null | wc -l | tr -d '
')"
```

```
echo "LANG: $(cat package.json 2>/dev/null | grep -o '"typescript"' | head -1 ||
echo 'unknown')"
```

输出类似:

PROJECT: my-app

BRANCH: feature/auth

CHANGED\_FILES: 5

LANG: typescript

Claude 读取这些信息后，审查时就知道：这是一个 TypeScript 项目，当前在 feature/auth 分支，

### ### 10.5 Preamble 的注意事项

- **\*\*快速\*\***: Preamble 会阻塞 Skill 的启动。太慢会影响体验。
- **\*\*安静\*\***: 不要输出太多。只输出 Claude 需要的信息。
- **\*\*容错\*\***: 用 `|| echo 'default'` 处理命令失败的情况。

> **\*\*溜哥的经验\*\***:

>

> 我写过一个 Preamble 太重了—跑了10秒。每次触发 Skill 都要等。后来精简到只收集必要信息，2

---

## ## §11 参考文件 (References)

### ### 11.1 为什么需要 References

SKILL.md 的正文是核心指令。但有时候，Claude 需要更多的上下文：

- 项目的代码规范
- API 文档
- 设计模式示例
- 配置模板

这些内容不适合放在 SKILL.md 里（太大），但 Claude 需要时得能看到。

References 解决这个问题。

### ### 11.2 References 的结构

my-skill/

├── SKILL.md

└── references/

├── code-style.md # 代码风格指南

├── api-docs.md # API 文档

├── examples/ # 示例目录

| ├── good.ts

| └── bad.ts

└── templates/ # 模板目录

└── component.tsx

### ### 11.3 在 SKILL.md 中引用

# 参考文件

在写代码前，先读取以下参考文件：

- 1. `references/code-style.md` — 了解项目的代码风格
- 2. `references/api-docs.md` — 了解可用的 API
- 3. `references/examples/good.ts` — 看一个好例子

根据这些参考文件来写代码。

```
Claude 读到这段话时，才会去加载这些文件。不读到就不会加载—这就是 Level 3 按需加载。

11.4 什么时候用 References

内容	放 SKILL.md	放 References
核心指令	✓	✗
工作流程	✓	✗
代码规范	✗	✓
API 文档	✗	✓
示例代码	✗	✓
配置模板	✗	✓

11.5 References 的大小管理

References 文件不要太大。建议：

- 单个文件 < 500 行
- 总量 < 2000 行
- 用目录分组，不要平铺

> **核心建议**：
>
> References 是 Skill 的"参考书架"。SKILL.md 告诉 Claude "怎么做"，References 告诉 C

§12 打包发布

12.1 什么时候发布

你的 Skill 满足以下条件时，考虑发布：

1. 你自己用了至少一周
2. 没有严重的 bug
3. 文档完整 (SKILL.md 写清楚了)
4. 对其他人也有用

12.2 发布到 npm
```

## 初始化 package.json

---

`npm init -y`

设置 package name

---

package.json 里加上：

---

**"name": "@yourname/skill-name"**

---

"keywords": ["claude-code-skill"]

---



## 发布

---

npm publish

发布后，其他人可以这样安装：

npx skills add @yourname/skill-name

### 12.3 发布到 GitHub

## 创建仓库

---

gh repo create my-skill --public

## 推送代码

---

```
git init
```

```
git add .
```

```
git commit -m "feat: initial release"
```

```
git push
```

其他人可以这样安装：

```
npx skills add yourname/my-skill
```

```
12.4 _metadata.json
```

发布前，建议加上元数据文件：

```
{

 "version": "1.0.0",

 "author": "yourname",

 "license": "MIT",

 "repository": "https://github.com/yourname/my-skill",

 "description": "A skill for..."
}
```

### ### 12.5 发布清单

- [ ] SKILL.md 完整 (name、description、正文)
- [ ] 测试过至少一周
- [ ] README.md 说明安装和使用
- [ ] \_metadata.json 包含版本信息
- [ ] 没有硬编码的路径或密钥
- [ ] 脚本有错误处理

> \*\*溜哥的经验\*\*:

>

> 我第一个发布的 Skill, README 写了3行。结果没人用。后来加了安装说明、使用示例、截图, 才有人

---

### ## Part 4: 进阶技巧

掌握这些技巧, 你能用 Skills 做更复杂的事: 条件触发、多 Skills 协作、性能优化。

---

### ## §13 条件触发和上下文感知

#### ### 13.1 Skills 怎么感知上下文

Claude 在触发 Skill 之前, 已经有了当前对话的上下文: 你在做什么项目、改了什么文件、之前说了什

Skill 可以利用这些上下文。

#### ### 13.2 在 SKILL.md 中写条件逻辑

## 工作流程

---

### 1. 检查当前项目类型

- 如果是 React 项目: 使用 React Testing Library
- 如果是 Vue 项目: 使用 Vue Test Utils
- 如果是 Node 项目: 使用 Jest 直接测试

### 1. 检查测试框架

- 如果已有 Jest 配置: 使用 Jest
- 如果已有 Vitest 配置: 使用 Vitest
- 如果没有测试框架: 推荐 Jest

Claude 会根据项目的实际情况，选择对应的路径。

### 13.3 利用 Preamble 提供上下文

```
#!/bin/bash
```

## 检测项目类型

---

```
if [-f "package.json"]; then

if grep -q "react" package.json; then

echo "PROJECT_TYPE: react"

elif grep -q "vue" package.json; then

echo "PROJECT_TYPE: vue"

else

echo "PROJECT_TYPE: node"

fi

fi
```

# 检测测试框架

```
if [-f "jest.config.js"] || [-f "jest.config.ts"]; then

echo "TEST_FRAMEWORK: jest"

elif [-f "vitest.config.ts"]; then

echo "TEST_FRAMEWORK: vitest"

else

echo "TEST_FRAMEWORK: none"

fi
```

```
13.4 条件触发的最佳实践

场景	做法
项目类型不同	用 Preamble 检测, 在 SKILL.md 写分支
文件类型不同	用 Claude 的上下文判断
用户偏好不同	用 CLAUDE.md 定义偏好

> **核心建议**:
>
> 不要写一个大而全的 Skill。写多个小 Skill, 每个处理一个场景。Claude 会自动选择最相关的那个

§14 多 Skills 协作

14.1 串联执行

一个任务可能需要多个 Skills 依次完成。
```

/review → 发现问题 → /fix → 修复代码 → /test → 跑测试

```
你可以手动串联, 也可以写一个"编排 Skill"。

14.2 编排 Skill
```

name: full-check

description: Run full code quality check: review, fix, test





## Full Check

---

执行完整的代码质量检查流程：

1. 运行 code-reviewer Skill 审查代码
2. 根据审查结果修复问题
3. 运行 test Skill 确认测试通过
4. 生成质量报告

### ### 14.3 并行执行

有些 Skills 可以并行：

同时运行：

- /lint 检查代码风格
- /type-check 检查类型
- /test 跑测试

Claude Code 支持并行执行多个独立任务。

### ### 14.4 Skills 之间的通信

Skills 之间不直接通信。它们通过以下方式共享信息：

- **\*\*文件\*\***：一个 Skill 写文件，另一个读文件
- **\*\*Git\*\***：一个 Skill 提交代码，另一个读取 diff
- **\*\*上下文\*\***：都在同一个对话里，共享上下文

### ### 14.5 团队工作流示例

开发者日常：

1. 写代码
2. /tdd — 先写测试
3. /commit — 智能提交
4. /review — 自我审查
5. /ship — 发布

设计师日常：

1. /huashu-design — 生成设计

2. /review — 审查设计
3. /commit — 提交设计文件

运维日常：

1. /investigate — 排查问题
2. /fix — 修复
3. /deploy — 部署
4. /canary — 监控

```
> **滔哥的经验**:
>
> 多 Skills 协作的关键是"接口清晰"。每个 Skill 的输入输出要明确。我通常会在 SKILL.md 里写

§15 Skills 的性能优化

15.1 上下文是稀缺资源

Claude 的上下文窗口有限。每个 Skill 的 SKILL.md 加载进来，都会占用上下文。

如果一个 Skill 的 SKILL.md 有 2000 行，每次触发就占掉大量上下文。留给实际任务的空间就少了。

15.2 优化策略

策略一：精简 SKILL.md
```

核心指令 → SKILL.md

参考内容 → references/

示例代码 → references/examples/

配置模板 → assets/

SKILL.md 只放"怎么做"，不放"参考什么"。

**\*\*策略二：用 Preamble 预处理\*\***

不要在 SKILL.md 里写"先检查 package.json"

---

## 而是在 Preamble 里检查，输出结果

```
cat package.json | grep -o "react" | head -1
```

Preamble 的输出直接进入上下文，比在 SKILL.md 里写"请先检查"更高效。

**\*\*策略三：合理使用 model\*\***

model: haiku # 简单任务用 haiku

model: sonnet # 中等任务用 sonnet

model: opus # 复杂任务用 opus

简单任务不需要最强的模型。用 haiku 快 3 倍，便宜 10 倍。

**\*\*策略四：按需加载 Level 3\*\***

只有当用户要求生成报告时，才读取 `references/report-template.md`。

不要在 SKILL.md 开头就让 Claude 加载所有参考文件。

### ### 15.3 监控上下文使用

在对话中输入 `/cost``，查看当前上下文使用情况。如果发现上下文快满了：

1. `/clear`` 清除上下文
2. 检查哪些 Skill 加载了太多内容
3. 优化 SKILL.md 和 References

> **\*\*核心建议\*\***:

>

> 性能优化的第一步是测量。先看看你的 Skill 占了多少上下文，再决定怎么优化。不要盲目精简——有些

---

## §16 Skills 和 Agent、MCP 的配合

### ### 16.1 三种扩展方式的关系

Skills：怎么做事（方法论）

Agent：谁来做（执行者）

MCP：用什么工具（外部能力）

它们是三层：

Agent（任务层）

↓ 调用

Skills（方法层）

↓ 使用

MCP（工具层）

### ### 16.2 Skills + Agent

Agent 可以调用 Skills。一个“代码审查 Agent”可以：

1. 调用 /review Skill 审查代码
2. 调用 /fix Skill 修复问题
3. 调用 /test Skill 跑测试
4. 生成报告

### ### 16.3 Skills + MCP

Skills 可以使用 MCP 提供的工具。一个“部署 Skill”可以：

1. 用 GitHub MCP 创建 PR
2. 用 Vercel MCP 触发部署
3. 用 Slack MCP 发送通知

### ### 16.4 完整的工作流示例

用户：发布新版本

Agent（编排）

└── /commit Skill（提交代码）

└── /review Skill（审查代码）

└── GitHub MCP（创建 PR）

└── /test Skill（跑测试）

└── Vercel MCP（部署）

└── Slack MCP（通知团队）

### ### 16.5 配置建议

| 需求     | 方案                   |
|--------|----------------------|
| 单一任务   | 用 Skill              |
| 多步骤任务  | 用 Agent + Skills     |
| 需要外部能力 | Skill + MCP          |
| 完整流程   | Agent + Skills + MCP |

> **\*\*滔哥的经验\*\***:

>

> 刚开始我只用 Skills。后来发现有些任务太复杂，一个 Skill 搞不定。就开始写 Agent 来编排多个

---

## ## Part 5: Skills 生态全集

精选社区最实用的 Skills，按类别组织。读完这部分，你知道每个领域有什么好用的 Skill。

---

## ## §17 开发类 Skills

### ### 17.1 代码质量

| Skill         | 来源             | 功能     | 推荐度   |
|---------------|----------------|--------|-------|
| code-reviewer | alirezarezvani | 代码审查   | ★★★★★ |
| linter        | 社区             | 代码风格检查 | ★★★★  |
| formatter     | 社区             | 代码格式化  | ★★★★  |
| refactor      | 社区             | 代码重构   | ★★★★  |

**\*\*code-reviewer\*\*** 是最常用的开发类 Skill。它不只是检查语法错误，还会分析代码结构、命名规范

### ### 17.2 测试

| Skill     | 来源 | 功能     | 推荐度   |
|-----------|----|--------|-------|
| tdd       | 社区 | 测试驱动开发 | ★★★★★ |
| unit-test | 社区 | 单元测试生成 | ★★★★  |
| e2e-test  | 社区 | 端到端测试  | ★★★   |
| snapshot  | 社区 | 快照测试   | ★★★   |

**\*\*tdd\*\*** Skill 实现了真正的测试驱动开发流程：先写测试，再写代码，最后确认测试通过。

### ### 17.3 Git 和版本控制

| Skill  | 来源     | 功能     | 推荐度   |
|--------|--------|--------|-------|
| commit | 内置     | 智能提交   | ★★★★★ |
| ship   | gstack | 完整发布流程 | ★★★★★ |
| branch | 社区     | 分支管理   | ★★★★  |

**\*\*commit\*\*** 是每天都在用的 Skill。它分析你的改动，生成规范的 commit message。

### ### 17.4 开发类 Skill 使用建议

日常开发：

- 1. /tdd — 先写测试
- 2. 写代码
- 3. /commit — 智能提交
- 4. /review — 自查

发布前：

- 1. /review — 完整审查
- 2. /test — 跑测试
- 3. /ship — 发布

```
> **核心建议**:
>
> 开发类 Skills 不需要装太多。一个代码审查、一个测试、一个提交，就够了。装太多反而会让 Claude

§18 设计类 Skills

18.1 设计生成

Skill	来源	功能	推荐度
huashu-design	alchaincyf	HTML 原生设计	★★★★★
ui-builder	社区	UI 组件生成	★★★★
figma-export	社区	Figma 导出	★★★

huashu-design 是设计类 Skill 中最独特的一个。它不是生成设计稿，而是直接生成 HTML。你给

18.2 设计审查

Skill	来源	功能	推荐度
design-review	gstack	设计审查	★★★★★
accessibility	社区	可访问性检查	★★★★

18.3 设计类 Skill 的工作流
```

- 1. /huashu-design — 生成设计
- 2. 浏览器预览
- 3. /design-review — 审查设计
- 4. 迭代修改
- 5. /commit — 提交

```

> **溜哥的经验**:
>
> huashu-design 改变了我的设计 workflow。以前用 Figma 画设计稿，再手动写 HTML。现在直接用 SK

§19 运维类 Skills

19.1 部署

Skill	来源	功能	推荐度
ship	gstack	完整发布流程	★★★★★
deploy	社区	部署脚本	★★★★
docker	社区	Docker 构建	★★★★
ci-cd	社区	CI/CD 配置	★★★

ship 是最完整的发布 Skill。它包含：代码审查 → 测试 → 提交 → 创建 PR → 通知团队。

19.2 监控和排查

Skill	来源	功能	推荐度
investigate	gstack	问题排查	★★★★★
canary	gstack	部署后监控	★★★★
log-analyzer	社区	日志分析	★★★

investigate 是排查问题的利器。输入错误信息，它会系统性地分析原因、定位代码、提出修复方案。

19.3 安全

Skill	来源	功能	推荐度
cso	gstack	安全审计	★★★★★
dependency-check	社区	依赖安全检查	★★★★
secrets-scanner	社区	密钥扫描	★★★★

19.4 运维类 Skill 使用建议

```

发布流程：

1. /ship — 完整发布
2. /canary — 监控部署
3. /investigate — 排查问题（如有）

安全流程：

1. /cso — 安全审计
2. /dependency-check — 检查依赖
3. /secrets-scanner — 扫描密钥



```

> **核心建议**:
>
> 运维类 Skills 是"平时不用, 用时救命"。建议提前装好 /investigate 和 /cso。出问题时再装就

§20 效率类 Skills

20.1 文档生成

Skill	来源	功能	推荐度
doc-generator	社区	文档生成	★★★★★
readme-writer	社区	README 生成	★★★★★
changelog	社区	变更日志	★★★
api-doc	社区	API 文档	★★★★★

20.2 代码转换

Skill	来源	功能	推荐度
migrate	社区	框架迁移	★★★★★
upgrade	社区	版本升级	★★★★★
convert	社区	语言转换	★★★

20.3 思维增强

Skill	来源	功能	推荐度
nuwa-skill	alchaincyf	思维蒸馏	★★★★★
brainstorm	社区	头脑风暴	★★★★★
decision	社区	决策分析	★★★

nuwa-skill 是最特别的效率类 Skill。它不是帮你做事, 而是帮你思考。输入一个人的思维方式,

20.4 效率类 Skill 使用建议

```

项目启动:

1. /doc-generator — 生成文档
2. /readme-writer — 写 README

日常开发:

1. /api-doc — 生成 API 文档
2. /changelog — 更新变更日志

思考辅助:

1. /nuwa-skill — 思维蒸馏
2. /brainstorm — 头脑风暴

```

> **滔哥的经验**:
>
> nuwa-skill 让我重新理解了 Skills 的边界。它不是"帮你做事"的工具，是"帮你思考"的工具。你可

Part 6: 实战场景

真实项目中的 Skills 用法。这部分综合运用前面学到的所有能力。

§21 场景一：用 Skills 搭建完整开发流程

21.1 需求

一个5人的前端团队，想用 Skills 标准化开发流程。每个人写代码的习惯不同，提交质量参差不齐，代码

21.2 Skill 清单

```

.claude/skills/

```

├── commit/ # 智能提交
| └── SKILL.md
├── review/ # 代码审查
| └── SKILL.md
├── tdd/ # 测试驱动
| └── SKILL.md
├── ship/ # 发布流程
| └── SKILL.md
└── investigate/ # 问题排查
 └── SKILL.md

```

```

21.3 每个 Skill 的核心逻辑

```

```

commit Skill:

```

1. 分析 git diff
2. 判断改动类型 (feat/fix/refactor/docs)
3. 生成规范的 commit message
4. 执行提交

**\*\*review Skill\*\*:**

1. 读取改动文件
2. 检查代码风格
3. 检查潜在 bug
4. 检查安全问题
5. 生成审查报告

**\*\*tdd Skill\*\*:**

1. 读取目标代码
2. 分析函数签名
3. 生成测试用例
4. 运行测试
5. 确认通过

### ### 21.4 实际效果

| 指标                | 使用前       | 使用后                                              |
|-------------------|-----------|--------------------------------------------------|
| commit message 质量 | "fix bug" | "fix: resolve race condition in auth middleware" |
| 代码审查时间            | 2小时/PR    | 20分钟/PR                                          |
| 测试覆盖率             | 40%       | 85%                                              |
| 发布时间              | 30分钟      | 5分钟                                              |

### ### 21.5 团队反馈

- 开发者A: "以前写测试是最痛苦的事, 现在 /tdd 直接生成, 我只用检查。"
- 开发者B: "commit message 终于不用猜了。"
- 技术负责人: "代码审查从'人肉检查'变成了'人+AI 检查', 质量上去了, 时间下来了。"

> **\*\*滔哥的经验\*\*:**

>

> 这个场景不是假设。我自己的团队就是这么做的。5个人用了2个月, 效果确实好。关键是 Skills 不是看

---

## ## §22 场景二: 从零构建一个 Skill

### ### 22.1 需求

做一个"API 文档生成" Skill。每次写完接口, 自动扫描代码, 生成 API 文档。

### ### 22.2 第一步: 定义 Skill 的行为

name: api-doc

description: >

Scan TypeScript/JavaScript code for API endpoints (Express, Fastify, Hono)  
and generate API documentation in Markdown format.

---

# API Doc Generator

---

扫描代码中的 API 端点，生成 API 文档。

## 工作流程

---

1. 扫描项目中的路由定义
2. 提取端点信息（路径、方法、参数、返回值）
3. 生成 Markdown 格式的 API 文档
4. 输出到 docs/api.md

## 规则

---

- 只扫描 Express/Fastify/Hono 的路由定义
- 参数类型从 TypeScript 类型推断
- 返回值类型从响应对象推断
- 生成的文档包含：路径、方法、参数、返回值、示例

```
22.3 第二步：写 Preamble
```

```
#!/bin/bash
```

## 检测使用的框架

---

```
if grep -q "express" package.json 2>/dev/null; then
echo "FRAMEWORK: express"

elif grep -q "fastify" package.json 2>/dev/null; then
echo "FRAMEWORK: fastify"

elif grep -q "hono" package.json 2>/dev/null; then
echo "FRAMEWORK: hono"

else
echo "FRAMEWORK: unknown"

fi
```

## 统计路由文件数

---

```
echo "ROUTE_FILES: $(find src -name "*.ts" -exec grep -l
"router\\.\\.\\app\\.\\.\\get\\.\\post\\.\\put\\.\\delete" {} \; 2>/dev/null | wc -l | tr -d ' ')"
```

### 22.4 第三步：添加参考文件

```
.claude/skills/api-doc/
|—— SKILL.md
|—— scripts/
| |—— preamble.sh
|—— references/
|—— express-patterns.md # Express 路由模式
|—— fastify-patterns.md # Fastify 路由模式
|—— doc-template.md # 文档模板
```

### 22.5 第四步：测试

## 在项目中测试

---

```
cd my-api-project
```

```
claude
```



## 触发 Skill

---

/api-doc

检查输出的文档是否准确。

### 22.6 第五步：迭代

根据测试结果调整：

- 路由扫描不准？调整正则表达式
- 参数类型不对？改进类型推断逻辑
- 文档格式不好看？修改模板

### 22.7 第六步：发布

## 写 README

---

```
echo "# API Doc Generator\n\nGenerate API documentation from code." >
README.md
```

## 发布

---

npm publish

```
> **核心建议**:
>
> 构建 Skill 的关键是"先让它能用，再让它好用"。第一个版本不需要完美。能扫描出80%的路由就够了

§23 场景三：团队协作中的 Skills

23.1 需求

一个分布式团队，3个城市、8个人。需要统一的代码规范和工作流程。

23.2 方案：共享 Skills 仓库
```

team-skills/ # 独立仓库

```
|—— .claude/
| |—— skills/
| |—— code-style/ # 代码风格
| |—— commit-convention/ # 提交规范
| |—— review-checklist/ # 审查清单
| |—— deploy-checklist/ # 部署清单
|—— README.md
|—— install.sh # 一键安装脚本
```

```
23.3 一键安装脚本
```

#!/bin/bash

## install.sh

---

```
SKILL_DIR=".claude/skills"
```

```
mkdir -p "$SKILL_DIR"
```

## 复制 Skills

---

```
cp -r skills/* "$SKILL_DIR/"

echo "✓ Skills installed to $SKILL_DIR"

echo "Available skills:"

ls "$SKILL_DIR"
```

团队成员只需要:

```
git clone https://github.com/team/team-skills.git /tmp/team-skills

cd your-project

/tmp/team-skills/install.sh
```

### 23.4 版本管理

Skills 仓库用 Git 管理版本:

# 更新 Skills

cd team-skills

git pull

./install.sh # 重新安装到项目

```
23.5 团队规范

规范	负责 Skill	作用
代码风格	code-style	统一缩进、命名、注释
提交规范	commit-convention	统一 commit message 格式
审查标准	review-checklist	统一审查清单
部署流程	deploy-checklist	统一部署步骤

23.6 实际效果

- 新人入职第一天就能用上团队的 Skills
- 不同城市的开发者用同一套规范
- 代码风格差异从"经常出现"降到"几乎没有"
- 部署错误从"每月2-3次"降到"几乎为零"

> **滔哥的经验**:
>
> 团队协作最大的挑战不是技术，是"统一"。8个人，8种写代码习惯。Skills 解决了这个问题——不是靠增

附录

附录 A: SKILL.md 模板库

代码审查模板:

```

name: code-reviewer

description: >

Review code for bugs, security issues, and best practices.

Analyzes code structure, naming conventions, and potential improvements.



# Code Reviewer

审查代码质量，发现问题并给出改进建议。

## 工作流程

1. 读取目标文件
2. 分析代码结构
3. 检查常见问题
4. 生成审查报告

## 检查清单

- ☐ 命名是否清晰
- ☐ 函数是否过长 (>30行)
- ☐ 是否有未处理的异常
- ☐ 是否有硬编码的值
- ☐ 是否有安全风险
- ☐ 是否有性能问题

## 输出格式

按严重程度排序：

- 严重：必须修复
- 警告：建议修复
- 建议：可以改进

**\*\*测试生成模板\*\*：**

name: test-generator



description: >

Generate unit tests for TypeScript/JavaScript functions.

Uses Jest with snapshot testing for React components.

---

# Test Generator

为目标代码生成单元测试。

## 工作流程

1. 读取目标代码
2. 分析导出的函数和类
3. 确定测试框架（Jest/Vitest）
4. 生成测试文件
5. 运行测试确认通过

## 规则

- 测试文件放在 `__tests__` 目录
- 测试函数名用 `should` 开头
- 每个函数至少3个测试用例
- 包含正常路径和边界情况

**\*\*部署检查模板\*\***:

name: deploy-checklist

description: >

Pre-deployment checklist for production releases.

Checks environment, dependencies, tests, and security.

# Deploy Checklist

---

部署前检查清单。

## 检查项

---

1. 测试：所有测试是否通过？
2. 依赖：是否有安全漏洞？
3. 环境变量：生产环境变量是否配置？
4. 数据库：是否需要迁移？
5. 回滚计划：出问题怎么回滚？
6. 监控：监控是否就绪？

## 输出

---

按检查项逐项报告，每项标记  或 。

---

### ### 附录 B: 常见问题和解决方案

| 问题           | 原因              | 解决方案                                 |
|--------------|-----------------|--------------------------------------|
| Skill 不触发    | description 不匹配 | 优化 description 关键词                   |
| 触发了错误的 Skill | description 太宽泛 | 让 description 更具体                    |
| 上下文爆了        | SKILL.md 太长     | 精简正文, 内容移 references/                |
| 脚本执行失败       | 权限问题            | <code>`chmod +x scripts/*.sh`</code> |
| Skill 之间冲突   | 同时触发            | 用 description 区分场景                   |
| 输出质量不好       | 指令不够详细          | 加示例和规则                               |
| 安装后找不到       | 目录位置错           | 检查 <code>`.claude/skills/`</code> 路径 |

---

### ### 附录 C: Skills 生态资源

| 资源            | 链接                                                                                                    | 说明            |
|---------------|-------------------------------------------------------------------------------------------------------|---------------|
| 官方文档          | <a href="https://code.claude.com/docs/en/skills">code.claude.com/docs/en/skills</a>                   | Skills 官方指南   |
| 社区 Skills 仓库  | <a href="https://github.com/alirezarezvani/claude-skills">github.com/alirezarezvani/claude-skills</a> | 235个 Skills   |
| Skills 市场     | <a href="https://skills.sh">skills.sh</a>                                                             | Skills 搜索和安装  |
| nuwa-skill    | <a href="https://github.com/alchaincyf/nuwa-skill">github.com/alchaincyf/nuwa-skill</a>               | 思维蒸馏          |
| huashu-design | <a href="https://github.com/alchaincyf/huashu-design">github.com/alchaincyf/huashu-design</a>         | 设计 Skill      |
| npm skills    | <a href="https://npmjs.com/search?q=claude-code-skill">npmjs.com/search?q=claude-code-skill</a>       | npm 上的 Skills |

---

### ### 附录 D: Skill 开发清单

#### \*\*第一步: 定义\*\*

- [ ] 明确 Skill 的用途
- [ ] 确定触发场景
- [ ] 设计工作流程

#### \*\*第二步: 编写\*\*

- [ ] 创建 SKILL.md
- [ ] 写 YAML 元数据 (name、description)
- [ ] 写工作流程 (编号列表)
- [ ] 写规则 (约束行为)
- [ ] 写示例 (输入输出)

#### \*\*第三步: 资源\*\*

- [ ] 添加 Preamble 脚本 (如需要)
- [ ] 添加 References 文件 (如需要)
- [ ] 添加 Assets 资源 (如需要)

#### \*\*第四步: 测试\*\*

- [ ] 在项目中测试
- [ ] 检查触发是否准确
- [ ] 检查输出质量
- [ ] 检查上下文使用

#### \*\*第五步: 发布\*\*

- [ ] 写 README.md

- [ ] 添加 \_metadata.json
- [ ] 发布到 npm 或 GitHub

---

## 写在最后

Skills 改变的不是 Claude Code 的能力，是 Claude Code 的专注度。

没有 Skills，Claude Code 是一个什么都会的通才。有了 Skills，它在每个领域都能变成专家。

这本书的23个章节，从"Skills 是什么"讲到"团队协作中的 Skills"。每章都建立在前一章的基础上。

我的建议：

1. **\*\*先用\*\***。安装几个社区 Skills，跑通流程
2. **\*\*再学\*\***。理解触发机制、三级加载、性能优化
3. **\*\*再写\*\***。从简单的 Skill 开始，逐步增加复杂度
4. **\*\*再分享\*\***。发布到社区，或者团队共享

Skills 是 Claude Code 生态中最容易上手、也最容易被低估的能力。

它不需要你会写代码（虽然会写更好）。它需要你描述"怎么做一件事"。

如果你能把一件事描述清楚，你就能写一个 Skill。

如果你能写一个 Skill，你就能让你的 Claude Code 变成任何领域的专家。

---

**\*\*滔哥\*\***

2026年5月4日

---

- > **\*\*Claude Code Skills 橙皮书\*\* · v1.0**
- > 基于 Claude Code Skills 生态（2026年5月）
- > 作者：滔哥
- > 最后更新：2026年5月4日